B04 | 两次 HTTP 交换怎样找到失败位置

本章把位置串起来：**调用者 → 本地 API → 供应商 → 本地 API → 调用者。** 这是两次 HTTP 交换；本地代码中间还通过 `return` 交值，不是一份回应从头穿到底。

谁接收请求，谁检查结果？

当前问题	关系或短用法	记到什么程度
本地 API 到底是 client 还是 server？	对最初调用者是 server，请求供应商时是 client。	**角色跟着交换判断。** 两份回应有不同接收者和内容约定。
发给供应商的内容从哪里来？	`ticket_id` 来自 path，`payload.text` 来自 body，关联标识来自 header；先确认工单存在，再发送本次 body 文字。	本例不自动读取工单全部内容；目标编号、待处理文字、关联标识各有用途。
header 怎样继续传给下游？	本例 `Header(default=None)` 接收可缺少的请求头，`x_request_id` 对应 `X-Request-ID`；取得值后放到 HTTPX 的 `headers`。	认熟“header → 参数 → 下游 header”；完整声明和编号生成查用。
供应商成功，结果怎样到最初调用者？	adapter 返回受检摘要字符串；endpoint 加入 `ticket_id`、`request_id`，交给输出 model，再形成外层回应。	**内部 return 交值，HTTP response 跨进程交消息。** 内层 200 不单独证明外层成功。
本地输入不合格，供应商会收到吗？	文字长度不合格时 422，endpoint 未开始；工单不存在时 404，supplier 调用尚未发生。	从最早停止的位置判断后续是否发生，不把所有失败归给供应商。

关联标识怎样回到成功、失败、输入校验回应，处理关系查原章：普通 `response.headers[...]` 不会自动替另一份 `HTTPException` 回应补 header；422 又可能发生在 endpoint 之前。需要改 422 处理器时，完整实现查配套 `app.py` 的 `validation_error_handler`，流程说明不等于完整代码。

回看：「本地 API 既接收请求，也会发出请求」「Header 怎样进入函数，再进入下一次请求」「本地 API 先决定输入能否继续」「成功时，有两份回应和一次本地返回」「空文字为什么只能得到一段 HTTP 交换」。

看见失败状态，先说清它属于哪一段

当前问题	本章应用的失败路线	记到什么程度
供应商 500，为什么调用者看见 502?	内层 500 → `SupplierBadStatus` → 本地 endpoint 选择外层 502，`detail` 是 `supplier_bad_status`。	**两份回应、两个状态。** 不是 HTTPX 自动改码，也不是所有应用必须这样映射。
外层有 504，为什么又说没有完整回应?	内层读取超时，未取得完整 Response；本地接住 `SupplierTimeout`，另给外层 504。	带上观察位置；**超时不证明供应商没执行，也不撤销其工作。**
连不上，与读取超时相同吗?	其他请求失败走 `SupplierUnavailable`，本地回答 503；超时走 504。	保留原因，不只记“网络错了”；尝试调用不证明对方已收到。
供应商 200，外层也可能 502 吗?	媒体类型、JSON 或摘要字段不合约 → `SupplierBadContent` → 本地 502，`detail` 是 `supplier_bad_content`。	同为 502，断点可能不同；结合 detail 与日志判断。

本例用假供应商延迟回应形成 read timeout；具体等待秒数是受控配置，不用背，也不代表整条业务路线的总限时。

回看：「供应商已经返回500，本地为什么选择回答502」「超时路线缺的是供应商完整回应，本地仍然可以回答504」「连接失败和坏内容也要保留自己的原因」。

怎样把证据对应回这一次处理?

当前问题	关系或短用法	记到什么程度
`request_id` **与工单 id 一样吗?**	工单 id 指业务记录；request_id 关联一次处理。同次跨服务工作沿用，新的一次外部请求使用新标识。	**关联不证明成功，不授予权限，也不自动防重复。** 示例编号不背。
`supplier_call_started` **就证明对方收到吗?**	它表示本地进入调用位置；供应商的 `supplier_request_received` 再补接收证据。`supplier_call_succeeded` 表示 adapter 拿到受检摘要，仍在外层输出前。	按事件位置读：开始、内层成功、外层完成不是同一件事。
浏览器 Network 能看全程吗?	它看到浏览器的外层交换，看不到本地 Python 自己发出的内层请求；用 request_id 再追服务日志。	先找观察者的位置；仅有 ID 没有事件与结果，还不能还原经过。
核对零调用、参数与公开错误，用什么?	记录型 supplier 替身接在 endpoint 后，保存调用或抛指定异常；已有历史记录时看本次是否新增。	它检查函数交接；B03 的 MockTransport 检查 adapter 的 HTTP 处理，两者替换位置不同。
替身通过，能证明两个服务接通吗?	不能；两服务实际启动并交换消息才增加跨进程证据。假供应商按固定规则回答，不是真实模型。	**各证据只证明自己的范围。** 本机 fake 通过不扩大为生产或客户环境已验证。

日志保留阶段、标识和必要状态，不靠保存 token 或整段敏感正文换取关联能力。INFO 事件还需实际日志配置；屏幕没显示，不能单独证明没调用。例如外层 502 与同 ID 的 `supplier_status=500` 合起来，才能找到“内层已有回应，在状态检查处停止”。

回看：「两条请求交错时，怎样知道一行日志属于哪次工作」「日志记录阶段，使处理经过可以被读回」「最初调用者的 Network 能看到哪一段」「先在函数交接处固定结果，再经过两个真实服务」「真实双服务验证要把不同请求分别认出来」。

本章留下的是 **哪段交换、最后成立的事实、最早失败的位置**；B05 把这些用于一次完整使用。